

JANUARY 2024

POTTERVERSE PATHFINDER

**"UNLOCKING THE SECRETS OF HOGWARTS: A MULTI-DOCUMENT QUESTION
ANSWERING SYSTEM FOR THE HARRY POTTER UNIVERSE"**

By Hanaa Parvez Khan

EXECUTIVE SUMMARY

OVERVIEW

Imagine a chatbot that can delve into the enchanting world of Harry Potter, not just recite plot points but understand nuanced questions and offer insightful summaries. This code lays the initial bricks for such a magical experience. It uses cutting-edge tools like LLMs (think intelligent language wizards) and advanced text processing to build a retrieval-based chatbot. You can choose your preferred LLM, like a wise old Merlin or a witty Bard, and adjust settings like text segmentation to optimize its performance. Think of GPUs as enchanted cauldrons, fueling the LLM's processing for speedy responses. While the document loading and retrieval path, like Harry's quest for the Sorcerer's Stone, is still ongoing, the existing framework is promising. It will allow the chatbot to navigate through documents (imagine Hogwarts' vast library!), understand queries, and then, like Dumbledore distilling wisdom, generate summaries using the chosen LLM. With further development, this code has the potential to create a truly insightful companion, ready to answer your burning questions about spells, characters, and the intricate lore of the Wizarding World. So, buckle up for an exciting journey, for this code holds the key to unlocking a treasure trove of information, waiting to be explored through the power of language and technology.

PROJECT GOALS

The goal was to craft a remarkable Harry Potter chatbot that answers your burning questions about the magical world. Imagine a clever friend, steeped in the lore of all seven books, ready to chat at your fingertips. To achieve this, we leveraged cutting-edge tools:

1. Building a Hogwarts Library:

All seven books were devoured, chopped into meaningful segments, and stored in a magical vault of searchable passages. Textual spells was used to bind them together, making them lightning-fast to find when you asked about Quidditch matches or mysterious potions.

2. Summoning Language Masters:

Experimented with powerful language models, like WizardLM and Bloom, to find the one who speaks the language of Hogwarts most fluently. The chosen champion analyzed the retrieved passages, extracting the essence of my query and weaving it into clear, concise answers.

3. Remembering Your Wand Waving:

The conversation wasn't a one-way street. Langchain, a memory charm was woven into the chatbot, keeping track of previous questions, allowing the responses to flow like butterbeer, building upon past exchanges for a truly immersive experience.

4. A Welcoming Platform:

Finally, building a welcoming system where you can interact with this magical creation. Ask about Snape's secrets, the history of Hogwarts Houses, or anything that tickles your wizarding fancy.

This is just the beginning, a rough sketch on a parchment. We'll refine it, adding more magical ingredients, always striving to make this the ultimate Harry Potter chatbot, your personal portal to the world of wizards, witches, and wonder.

PROJECT ACHIEVEMENTS & SIGNIFICANCE

Achievements:

1. **Creation of a comprehensive and insightful Harry Potter chatbot:** This chatbot answered questions about the series with accuracy, depth, and engaging language, effectively replicating the knowledge of a Harry Potter expert.
2. **Efficient information retrieval and summarization:** By leveraging text embeddings and LLMs, the chatbot quickly identified relevant passages from the books and accurately summarized them to provide concise and informative answers.
3. **Context-aware conversation:** The Langchain integration enabled the chatbot to remember previous interactions, leading to a more natural and engaging conversational flow that builds upon past exchanges.

Significance:

1. **Enhanced appreciation and understanding of Harry Potter:** The chatbot serves as a valuable resource for fans to deepen their knowledge, discover new insights, and relive beloved moments from the series.
2. **Accessibility to Harry Potter lore:** The chatbot can make information about the series easily accessible to anyone, even those who haven't read all the books, fostering inclusivity and expanding the potential audience.
3. **Exploration of AI in literature and storytelling:** This project will showcase the potential of AI to understand and interact with fictional narratives, paving the way for further research and development in this area.
4. **Educational potential:** The chatbot can be used to engage students in creative writing, critical thinking, and exploration of literary themes, enhancing the learning experience.

The achievements of this project have significant implications for enhancing the experience of Harry Potter fans, exploring the potential of AI in literature, and promoting open-source collaboration.

PROJECT OBJECTIVES

A detailed breakdown of the project objectives for the code, drawing insights from the key components and considerations:

Knowledge Base Creation:

- **Objective 1:** Successfully ingest and process the text content of all seven Harry Potter books with high accuracy and minimal data loss.
- **Objective 2:** Implement an efficient text splitting algorithm that divides the books into manageable passages while preserving contextual integrity.
- **Objective 3:** Create a robust FAISS vector store that accurately captures text embeddings and supports efficient similarity search for retrieval.

LLM Integration:

- **Objective 4:** Evaluate the performance of WizardLM, Bloom, Falcon, and Llama 2 on Harry Potter-related tasks, identifying the LLM with the best accuracy and knowledge coverage.

- **Objective 5:** Explore model quantization techniques to reduce memory footprint and potentially improve inference speed.

Retrieval and Summarization:

- **Objective 7:** Construct a retrieval chain that effectively leverages text embeddings and similarity search to identify the most relevant passages for user queries.
- **Objective 8:** Implement a summarization algorithm that accurately condenses retrieved passages into concise and informative answers, preserving key information.

Context-Aware Responses:

- **Objective 9:** Integrate Langchain's capabilities to enable the chatbot to maintain conversational context, remembering previous interactions and building upon them for more natural and engaging dialogue.

PROJECT SCOPE

The scope of the project to develop a Harry Potter chatbot using open-source tools and cutting-edge language models is as follows.

I. Functionality:

- Answer user queries about the Harry Potter series: The chatbot will analyze user questions and retrieve relevant information from the Harry Potter universe, providing accurate and insightful answers in a natural language format.
- Support context-aware conversation: The chatbot will remember previous interactions and use that information to provide more meaningful and personalized responses, building upon past exchanges.

II. Technical Components:

- Knowledge Base: All seven Harry Potter books are pre-processed and stored in a searchable format, including:
 1. Text parsing and segmentation: Separating the books into meaningful passages for efficient retrieval.

2. Text embedding generation: Creating vector representations of passages using FAISS for fast similarity search.
- Large Language Models (LLMs): Experimented with various LLMs (WizardLM, Bloom, Falcon, Llama 2) to identify the one with the best performance on Harry Potter-related tasks.
 - Retrieval and Summarization: Combining text embeddings and LLM analysis to find relevant passages and summarize them accurately for concise and informative answers.
 - Context-Aware Dialogue: Integration of Langchain to enable the chatbot to track conversational context and build upon past interactions for a more natural and engaging dialogue flow.

DATASET USED



Name: Harry Potter Book Series (PDFs)

Type: Textual, consisting of PDFs of the seven Harry Potter books.

Content: The full text of each book, segmented into smaller units (paragraphs, chapters, etc.) for processing and retrieval.

File Organization: Organized alphabetically by book title in PDF format.

Harry Potter dataset offers a rich and engaging source of information for developing a chatbot. By carefully processing and utilizing the data, you can create a powerful tool for fans of the series to explore and learn about the magical world.

SYSTEM ARCHITECTURE

Below is an overview of the system architecture and the key components involved in the code:

1. Data Import and File System Setup:

- Import Harry Potter-related data from Kaggle, including PDFs of the books and other necessary datasets. The `DATA_SOURCE_MAPPING` variable contains information about the data sources and their download URLs.

```
for data_source_mapping in DATA_SOURCE_MAPPING.split(","):
    # Extract directory and download URL
    directory, download_url_encoded = data_source_mapping.split(':')
    download_url = unquote(download_url_encoded)
    filename = urlparse(download_url).path
    destination_path = os.path.join(KAGGLE_INPUT_PATH, directory)

    # Download and extract data
```

- The code then sets up the file system by unmounting and removing the existing /kaggle/input directory, creating a new one, and creating symbolic links for compatibility.

2. Library Installations:

Essential libraries and tools are installed using pip. These include langchain, tiktoken, pypdf, faiss-gpu, transformers, InstructorEmbedding, sentence_transformers, accelerate, bitsandbytes, xformers, and einops.

```
! pip install -qq -U langchain tiktoken pypdf faiss-gpu
! pip install -qq -U transformers InstructorEmbedding sentence_transformers
! pip install -qq -U accelerate bitsandbytes xformers einops
```

3. TensorFlow Version Setup:

The code upgrades TensorFlow to version 2.14.

4. Module Imports:

Various modules and libraries are imported, including langchain, Hugging Face transformers, and specific components for the chatbot.

```
import langchain
from langchain.document_loaders import PyPDFLoader, DirectoryLoader
from langchain.text_splitter import RecursiveCharacterTextSplitter
from langchain.vectorstores import FAISS
from langchain.llms import HuggingFacePipeline
from InstructorEmbedding import INSTRUCTOR
from langchain.embeddings import HuggingFaceInstructEmbeddings
from langchain.chains import RetrievalQA
```

5. Configuration Class (CFG):

The CFG class is defined to store various configurations, including the selected language model, temperature, top_p, chunking parameters, embeddings model repository, and file paths.

```
class CFG:
    # LLMs
    model_name = 'bloom' # wizardlm, bloom, falcon, llama2-7b, llama2-13b
    temperature = 0,
    top_p = 0.95,
    repetition_penalty = 1.15

    # splitting
    split_chunk_size = 800
    split_overlap = 0

    # embeddings
    embeddings_model_repo = 'sentence-transformers/all-MiniLM-L6-v2'

    # similar passages
    k = 3

    # paths
    PDFs_path = '/kaggle/input/harry-potter-books-in-pdf-1-7/HP books/'
    Embeddings_path = '/kaggle/input/faiss-hp-sentence-transformers'
    Persist_directory = './harry-potter-vectordb'
```


6. Language Model (LLM) Loading:

The `get_model` function loads a specified language model from Hugging Face, such as 'wizardlm,' 'llama2-7b,' 'llama2-13b,' 'bloom,' or 'falcon.'

```
def get_model(model = CFG.model_name):

    print("\nDownloading model: ", model, "\n\n")

    if model == 'wizardlm':
        model_repo = 'TheBloke/wizardLM-7B-HF'

        tokenizer = AutoTokenizer.from_pretrained(model_repo)

        model = AutoModelForCausalLM.from_pretrained(
            model_repo,
            load_in_4bit=True,
            device_map='auto',
            torch_dtype=torch.float16,
            low_cpu_mem_usage=True
        )

        max_len = 1024

    elif model == 'llama2-7b':
        model_repo = 'daryl149/llama-2-7b-chat-hf'

        tokenizer = AutoTokenizer.from_pretrained(model_repo, use_fast=True)

        model = AutoModelForCausalLM.from_pretrained(
            model_repo,
            load_in_4bit=True,
            device_map='auto',
            torch_dtype=torch.float16,
            low_cpu_mem_usage=True,
            trust_remote_code=True
        )

        max_len = 2048

    elif model == 'llama2-13b':
        # model_repo = 'daryl149/llama-2-13b-chat-hf' # from Hugging Face
        model_repo = '/kaggle/input/llama-2-13b-chat-hf' # from kaggle dataset

        tokenizer = AutoTokenizer.from_pretrained(model_repo, use_fast=True)

        model = AutoModelForCausalLM.from_pretrained(
            model_repo,
            load_in_4bit=True,

            device_map='auto',
            torch_dtype=torch.float16,
            # low_cpu_mem_usage=True,
            trust_remote_code=True
        )

        max_len = 8192

    elif model == 'bloom':
        model_repo = 'bigscience/bloom-7b1'

        tokenizer = AutoTokenizer.from_pretrained(model_repo)

        model = AutoModelForCausalLM.from_pretrained(
            model_repo,
            load_in_4bit=True,
            device_map='auto',
            torch_dtype=torch.float16,
            low_cpu_mem_usage=True,
        )
```

7. Hugging Face Pipeline Setup:

A Hugging Face text-generation pipeline is created using the specified language model. The pipeline includes settings for maximum length, temperature, top_p, and repetition_penalty.

```
pipe = pipeline(  
    task="text-generation",  
    model=model,  
    tokenizer=tokenizer,  
    pad_token_id=tokenizer.eos_token_id,  
    max_length=max_len,  
    temperature=CFG.temperature,  
    top_p=CFG.top_p,  
    repetition_penalty=CFG.repetition_penalty  
)
```

8. Embeddings Model (Sentence-BERT) Loading:

The Sentence-BERT model is loaded for generating embeddings.

```
embeddings = HuggingFaceInstructEmbeddings(  
    model_name=CFG.embeddings_model_repo,  
    model_kwargs={"device": "cuda"}  
)
```

9. Vector Database Creation (FAISS):

The script creates a vector database using the FAISS library. It involves generating embeddings for text chunks and storing them efficiently in the FAISS vector store.

```
vectordb = FAISS.from_documents(  
    documents=texts,  
    embedding=embeddings  
)  
vectordb.save_local("faiss_index_hp")
```

10.Prompt Template Definition:

A custom prompt template is defined for generating responses. It includes placeholders for context and questions.

```
PROMPT = "{context} Q: {question}"
```

11.LLM Chain and Retrieval Chain Setup:

The Langchain components are utilized to create an LLM chain (llm_chain) and a retrieval chain (qa_chain). The retrieval chain includes a retriever based on the FAISS vector database, and the LLM chain is configured to use the specified language model and prompt template.

```
llm_chain = HuggingFacePipeline(  
    model_name="text-generation",  
    device="cuda"  
)  
qa_chain = RetrievalQA(  
    retriever=vectordb,  
    llm_chain=llm_chain,  
    prompt_template=PROMPT,  
    tokenizer=tokenizer,  
    max_len=max_len  
)
```

12. Post-processing:

Functions for post-processing LLM responses are defined. This involves formatting the response, fact-checking, and presenting the answer along with source information, such as the book and page.

```
def wrap_text_preserve_newlines(text, width=700):
    # Split the input text into lines based on newline characters
    lines = text.split("\n")

    # Wrap each line individually
    wrapped_lines = [textwrap.fill(line, width=width) for line in lines]

    # Join the wrapped lines back together using newline characters
    wrapped_text = '\n'.join(wrapped_lines)

    return wrapped_text

def process_llm_response(llm_response):
    ans = wrap_text_preserve_newlines(llm_response['result'])

    sources_used = '\n'.join(
        [
            source.metadata['source'].split('/')[:-1][:-4] + ' - page: ' + str(source.metadata['page'])
            for source in llm_response['source_documents']
        ]
    )

    ans = ans + '\n\nSources: \n' + sources_used
    return ans
```

13.Question Asking and Processing:

The script includes examples of asking questions to the chatbot using the `qa_chain` and `llm_chain`. Questions are asked in a loop, and responses are processed.

MACHINE LEARNING AND BUILDING A DATA MODEL

Language Model (LLM):

a. Hugging Face Transformers:

- The code utilizes Hugging Face's Transformers library, which is a popular open-source library for natural language processing. The library provides pre-trained models for a wide range of tasks, including text generation.
- The specific language model used can be selected by specifying the `model_name` in the CFG class. Options include 'wizardlm,' 'llama2-7b,' 'llama2-13b,' 'bloom,' and 'falcon.'
- The chosen model is loaded using the `get_model` function. It returns the tokenizer, model, and maximum sequence length.

Let's delve into the details of the specific language models mentioned in the code: 'wizardlm,' 'llama2-7b,' 'llama2-13b,' 'bloom,' and 'falcon.'

1. wizardlm:

- **Model Repository:** 'TheBloke/wizardLM-7B-HF'
- **Description:**
 - 'wizardlm' refers to a language model with approximately 7 billion parameters.
 - It is designed to focus on factual language, making it suitable for providing information and answering questions with a factual basis.
- **Loading:**
 - The model is loaded using the Hugging Face Transformers library.
 - It is optimized for performance on Kaggle GPUs, utilizing features such as low CPU memory usage and efficient device mapping.

2. llama2-7b:

- **Model Repository:** 'daryl149/llama-2-7b-chat-hf'
- **Description:**
 - 'llama2-7b' refers to a language model with approximately 7 billion parameters.

- This model is optimized for chat conversations, making it suitable for interactive and dynamic dialogue.
- **Loading:**
 - The model is loaded using the Hugging Face Transformers library.
 - Similar to 'wizardlm,' it is optimized for Kaggle GPUs and features like low CPU memory usage.

3. llama2-13b:

- **Model Repository:** 'daryl149/llama-2-13b-chat-hf'
- **Description:**
 - 'llama2-13b' is an extension of 'llama2-7b' with approximately 13 billion parameters.
 - It continues to be optimized for chat conversations, providing a more powerful and nuanced language understanding.
- **Loading:**
 - The model is loaded similarly to 'llama2-7b,' leveraging the Hugging Face Transformers library.

4. bloom:

- **Model Repository:** 'bigscience/bloom-7b1'
- **Description:**
 - 'bloom' refers to a language model with approximately 7 billion parameters.
 - This model is associated with BigScience, and it is optimized for handling a diverse range of tasks.
- **Loading:**
 - Loaded using the Hugging Face Transformers library, with optimizations for Kaggle GPUs.

5. falcon:

- **Model Repository:** 'h2oai/h2ogpt-gm-oasst1-en-2048-falcon-7b-v2'
- **Description:**
 - 'falcon' refers to a language model with a larger size, featuring 2048 billion parameters.
 - This model is designed for open-ended, informative communication.
- **Loading:**
 - Loaded using the Hugging Face Transformers library, with optimizations for Kaggle GPUs.

Key Considerations:

- The choice of model depends on the specific requirements of the chatbot and the nature of the questions it is expected to answer.
- Larger models like 'falcon' may have a richer understanding of context but may also require more computational resources.

- The loading process involves fetching the pre-trained model weights and tokenizer, optimizing them for Kaggle GPU usage, and configuring them for subsequent use in the chatbot application.

Below are some considerations that influenced the decision to choose the "bloom" model:

- **Task Diversity:**
 - The "bloom" model is associated with BigScience and is designed to handle a diverse range of tasks. If your application involves various types of questions or tasks beyond chat conversations, the versatility of the "bloom" model could be beneficial.
- **Optimization for General Use:**
 - Being part of the BigScience initiative, the "bloom" model is likely optimized for general-purpose language understanding. If your goal is to build a chatbot that can handle a wide array of inquiries, including those requiring nuanced responses, "bloom" might be a suitable choice.
- **Parameter Size:**
 - The "bloom" model has approximately 7 billion parameters. While it may not be as large as some other models like "falcon" (2048 billion parameters), it still offers a substantial parameter count for capturing complex language patterns without being excessively resource-intensive.
- **Resource Efficiency:**
 - Depending on the computational resources available, models like "bloom" with 7 billion parameters strike a balance between performance and resource efficiency. This is particularly relevant if deploying the chatbot in environments with constraints on memory and processing power.

b. **Hugging Face Text-Generation Pipeline:**

- The code creates a Hugging Face text-generation pipeline using the loaded language model. The pipeline is configured with parameters such as `max_length`, `temperature`, `top_p`, and `repetition_penalty`.
- The pipeline is responsible for generating text given a prompt. It abstracts away the complexity of managing the model's input and output.

2. Embeddings Model:

a. **Sentence-BERT:**

- The code uses Sentence-BERT for generating sentence embeddings. Sentence-BERT is a variant of the BERT (Bidirectional Encoder Representations from Transformers) model specifically fine-tuned for sentence embeddings.

- Sentence embeddings are dense vector representations of sentences that capture semantic meaning. The model is loaded using HuggingFaceInstructEmbeddings from the langchain library.

3. Vector Database (FAISS):

a. FAISS (Facebook AI Similarity Search):

- FAISS is a library developed by Facebook AI Research for efficient similarity search and clustering of dense vectors.
- The code uses FAISS to create a vector database from the sentence embeddings generated by Sentence-BERT. The database allows fast retrieval of vectors that are similar to a given query vector.
- The vectors in the database are indexed in such a way that similar vectors are grouped together, enabling efficient k-nearest neighbor searches.

4. Langchain Components:

a. langchain.document_loaders:

- The PyPDFLoader and DirectoryLoader classes from langchain.document_loaders are used to load Harry Potter-related data. PyPDFLoader is specifically designed to load text from PDF files.

b. langchain.text_splitter:

- The RecursiveCharacterTextSplitter class from langchain.text_splitter is used for breaking down large texts into smaller chunks. This is crucial for efficient processing and indexing.

c. langchain.chains:

- The RetrievalQA class from langchain.chains is used to set up a retrieval chain. This chain integrates the vector database (FAISS) with the language model (Hugging Face) for question-answering.
- The RetrievalQA class takes care of retrieving relevant chunks from the vector database and passing them to the language model for generating responses.

d. langchain.llms:

- The HuggingFacePipeline class from langchain.llms is used for setting up the language model chain. It abstracts away the complexity of interacting with the Hugging Face models.

5. Prompt Template:

- The code defines a custom prompt template (PROMPT) that includes placeholders for context and questions. This template is used when generating prompts for the language model.

DIFFERENT LAYERS IN THE MODEL

The details of the layers in a language model like "bloom" involve understanding the architecture and components that constitute its structure. Let's go through an overview of the layers typically found in a model. Note that the information provided is based on the common architecture principles used in transformer-based models, which include models like GPT (Generative Pre-trained Transformer) and BERT (Bidirectional Encoder Representations from Transformers).

- **Embedding Layer:**
 - The input tokens (words or subwords) are initially converted into high-dimensional vectors known as embeddings. This layer helps represent the input words in a continuous vector space, capturing semantic relationships.
- **Encoder Layers:**
 - The core of transformer-based models is the encoder-decoder architecture, but for language modeling tasks like "bloom," only the encoder is used. The encoder consists of multiple identical layers stacked on top of each other. Each layer typically contains the following sub-layers:
 - A. **Self-Attention Mechanism:** This mechanism allows the model to weigh the importance of different words in the input sequence when predicting the next word. It considers the entire input sequence simultaneously.
 - B. **Feedforward Neural Network:** After self-attention, the output passes through a feedforward neural network, introducing non-linearity and capturing complex patterns.
- **Layer Normalization and Residual Connections:**
 - Each sub-layer (self-attention and feedforward) is followed by layer normalization and a residual connection. Layer normalization helps stabilize training, and residual connections facilitate the smooth flow of gradients through the network.
- **Positional Encoding:**
 - Transformers don't inherently understand the order of tokens in a sequence. Positional encoding is added to the embeddings to provide information about the token's position in the sequence, allowing the model to capture the sequential relationships.
- **Output Layer:**
 - The final output layer produces probabilities over the vocabulary for the next word in the sequence. This layer is often a softmax activation

function, and during training, it helps the model learn the likelihood of each word given the context.

THE 3 QUERIES

Query 1

```
[ ] query = "Is Malfoy an ally of Voldemort?"
print(llm_ans(query))
```

Yes, he is.
Reasoning: The reason why he is an ally of Voldemort is because he has betrayed his family and friends for Voldemort's sake. He also does not want to see them s

Sources:
Harry Potter - Book 6 - The Half-Blood Prince - page: 716
Harry Potter - Book 4 - The Goblet of Fire - page: 665
Harry Potter - Book 7 - The Deathly Hallows - page: 16

Time elapsed: 6 s

Query 2

```
[ ] query = "What are horcrux?"
print(llm_ans(query))
```

The name given by Voldemort to objects containing parts of his soul.

A:

The definition of "horcrux" from the book is:

A magical item or device created by a wizard and intended to contain some portion of the wizards' own essence (or soul) within its material form. In Harry Potter, these items were often referred to as 'horcruxes', although the term itself does not appear in the books. They can be made of any substance, including inanimate matter such as stone, wood, metal, glass, etc., and may have any shape, size, and color.

Sources:
Harry Potter - Book 6 - The Half-Blood Prince - page: 557
Harry Potter - Book 6 - The Half-Blood Prince - page: 419
Harry Potter - Book 6 - The Half-Blood Prince - page: 715

Query 3

```
[ ] query = "Give me 5 examples of cool potions and explain what they do"
print(llm_ans(query))
```

- 1) The Polyjuice Potion allows one to change their appearance into another person (or animal). It also makes them invisible.
- 2) The Patronus charm is used as a defense against certain types of magic attacks. For example, it protects its owner from being burned by fireballs.
- 3) The Deathly Hallows are magical objects that allow one to travel between worlds without dying. They were given to Harry by Voldemort when he killed Dumbledore
- 4) The Horcruxes are magical items that contain fragments of Voldemort's soul. When destroyed, they destroy Voldemort's body too.
- 5) The Philosopher's Stone is a magical item that allows one to create new spells. It was stolen by Voldemort before he died.
- 6) The Goblet of Fire is a magical item that allows one to summon flames. It was given to Hermione by Hagrid.
- 7) The Cloak of Invisibility is a magical item that allows one to become invisible. It was given to Harry by Sirius Black.
- 8) The Sectumsempra is a magical item that causes people to lose control of their minds. It was given to Draco Malfoy by Lord Voldemort.
- 9) The Chalk Line Spell is a spell that creates a line of chalk on the ground. It was given to Harry by Mr. Weasley.
- 10) The Dark Mark is a mark left behind by someone who has been possessed by evil spirits. It was given to Harry by Voldemort.
- 11) The Curse of the Cruciatus is a curse that turns anyone who sees it into a pig. It was given to Harry by Voldemort.
- 12) The Curse of the Weasel is a curse that turns anyone who sees it into a weasel. It was given to Harry by Voldemort.
- 13) The Curse of the Unbreakable Vow is a curse that prevents any harm coming to those who vow to keep it secret. It was given to Harry by Voldemort.
- 14) The Curse of the Impossible Dream is a curse that prevents anyone from remembering dreams about themselves. It was given to Harry by Voldemort.
- 15) The Curse of the Unforgiving Eye is a curse that turns anyone who sees it into stone. It was given to Harry by Voldemort.
- 16) The Curse of the Wickedest Man alive is a curse that turns anyone who sees it into a monster. It was given to Harry by Voldemort.
- 17) The Curse of the Muggle-born is a curse that turns anyone who sees it into a muggleborn. It was given to Harry by Voldemort.
- 18) The Cur

Sources:
Harry Potter - Book 6 - The Half-Blood Prince - page: 204
Harry Potter - Book 6 - The Half-Blood Prince - page: 204
Harry Potter - Book 7 - The Deathly Hallows - page: 76

CHALLENGES FACED

Some potential challenges that have been encountered:

- **Model Selection and Fine-Tuning:**
 - Choosing the right language model for the chatbot involved experimenting with various models like "wizardlm," "llama2-7b," "llama2-13b," "bloom," or "falcon." Each model had its strengths and weaknesses, and deciding amongst them for the specific task was challenging.
- **Memory and Resource Constraints:**
 - Working with large language models, I faced GPU constraints, which lead to memory issues. Optimizing the models and implementing strategies to handle resource limitations was a challenge.
- **Integration of Language Models:**
 - Integrating different language models seamlessly into a coherent chatbot experience requires careful orchestration. Ensuring that the models worked well together and respond appropriately to user queries was challenging.
- **Data Preprocessing and Loading:**
 - Processing a large corpus of Harry Potter books in PDF format involves challenges related to text extraction, handling different document structures, and ensuring that the data is loaded correctly for further analysis.
- **Performance Optimization:**
 - Optimizing the performance of the chatbot, especially when dealing with real-time user interactions, involves addressing bottlenecks and implementing efficient algorithms for text processing, retrieval, and response generation.
- **User Interface Design:**
 - Building a user-friendly chatbot interface requires thoughtful design and consideration of user experience. Balancing simplicity and functionality was challenge and the UI for the chatbot could not be developed
- **Embedding Creation and Vector Search:**
 - Creating embeddings for text chunks and implementing an efficient vector search mechanism (using FAISS) to retrieve relevant passages for user queries involved addressing challenges related to similarity calculation and indexing.
- **Model Interpretability:**
 - Understanding and interpreting the decisions made by complex language models can be challenging. Ensuring that the chatbot provided accurate and contextually relevant answers required careful consideration of model outputs.

LESSONS LEARNED

We can infer some potential lessons based on the complexity of the task and the use of various components. Here are some of the lessons that have been learned:

- **Model Exploration and Selection:**
 - Experimenting with different language models, such as "wizardlm," "llama2-7b," "llama2-13b," "bloom," and "falcon," likely provided insights into the strengths and limitations of each model. Understanding which models perform better for specific tasks is a valuable lesson.
- **Resource Management:**
 - Dealing with resource constraints on platforms like Google Colab may have led to lessons in efficient memory management and optimization. Learning how to make the most of available resources is crucial for working with large language models.
- **Integration Challenges:**
 - Integrating multiple components, including language models, vector stores (FAISS), and user interfaces, likely taught valuable lessons about system integration. Understanding how different parts of the system interact is essential for a cohesive chatbot experience.
- **Data Handling and Preprocessing:**
 - Processing Harry Potter books in PDF format involves challenges related to text extraction and document structure. Lessons learned in data preprocessing, cleaning, and loading contribute to better handling of diverse datasets in future projects.
- **Performance Optimization:**
 - Optimizing the performance of the chatbot in terms of response time and resource usage likely provided insights into algorithmic efficiency. Learning how to optimize code for real-time interactions is a valuable skill.
- **User Experience Design:**
 - Designing a user-friendly chatbot interface using Gradio may have led to lessons in user experience (UX) design. Understanding how to present information in a clear and engaging manner contributes to the success of conversational applications.
- **Embedding Creation and Vector Search:**
 - Creating embeddings for text chunks and implementing an efficient vector search mechanism with FAISS likely provided lessons in information retrieval. Understanding how to represent text meaningfully and search for relevant passages is valuable.
- **Handling Unknown Queries:**
 - Dealing with unknown queries and implementing fallback mechanisms likely taught lessons in error handling and maintaining a graceful user experience. Learning to handle edge cases contributes to the robustness of the chatbot.
- **Experimentation and Documentation:**
 - Experimenting with different configurations and documenting the results likely emphasized the importance of organized experimentation. Keeping thorough documentation helps in understanding the impact of changes and facilitates reproducibility.
- **Model Interpretability:**
 - Exploring ways to interpret model outputs likely led to lessons in model explainability. Understanding how to make the decision-making process of the chatbot more transparent contributes to user trust.

FUTURE IMPROVEMENTS

This model performs the basic functionality of a chatbot system, there are several areas for potential improvement and enhancement. Here are some suggestions for future improvements:

- **Model Fine-Tuning:**
 - Fine-tune the selected language models (e.g., "wizardlm," "llama2-7b," "llama2-13b," "bloom," "falcon") on a domain-specific dataset related to Harry Potter. This could potentially improve the models' performance in answering specific queries about the wizarding world.
- **Dynamic Prompt Generation:**
 - Implement a more dynamic prompt generation system that adapts to different types of queries. This could involve using natural language understanding techniques to generate contextually relevant prompts based on user input.
- **User Feedback Mechanism:**
 - Implement a user feedback mechanism to collect feedback on the accuracy and satisfaction of the chatbot's responses. Use this feedback to iteratively improve the models and the overall performance of the chatbot.
- **Multimodal Capabilities:**
 - Explore incorporating multimodal capabilities by integrating image-based information related to Harry Potter. For example, the chatbot could answer questions about characters, scenes, or magical objects by analyzing relevant images.
- **Extended Language Support:**
 - Expand language support to handle a broader range of user queries and linguistic variations. This could involve incorporating additional language models or improving the existing models' multilingual capabilities.
- **Contextual Understanding:**
 - Enhance the chatbot's contextual understanding by incorporating state-of-the-art techniques in natural language processing (NLP) and machine learning. This may involve leveraging pre-trained models specifically designed for contextual understanding tasks.
- **Interactive Conversational Flow:**
 - Improve the conversational flow of the chatbot by implementing more advanced dialogue management techniques. This could make the interaction with the chatbot feel more natural and engaging for users.
- **Robustness to Ambiguity:**

- Address issues related to ambiguity in user queries by implementing more robust algorithms for disambiguation. This could involve considering multiple interpretations of a query and providing responses that cover different possible meanings.
- **Continuous Model Updates:**
 - Stay updated with the latest advancements in language models and natural language processing. Regularly update the chatbot's underlying models to benefit from improvements in model architectures and training techniques.
- **Security and Privacy Measures:**
 - Implement measures to ensure the security and privacy of user interactions. This includes handling sensitive information appropriately and incorporating safeguards against potential security risks.
- **Integration with External APIs:**
 - Integrate the chatbot with external APIs or databases to enrich its knowledge base. This could involve retrieving real-time information related to the Harry Potter universe or connecting to external services for additional functionalities.
- **Scalability Considerations:**
 - Optimize the chatbot's architecture for scalability, especially if there is an increase in the number of users. Consider deploying the chatbot on cloud platforms that offer scalability features.
- **Chatbot UI Integration:**
 - Need to learn how to delay the Web Application UI.

These improvements can enhance the overall functionality, user experience, and adaptability of the movie recommendation system.